

Vue Gadget Shop — Step 12

Informations

Notions	Appels API, Axios, json-server, service, onMounted
Niveau	Formatif
Travail	Individuel — sans aide IA

Objectif

Remplacer les données statiques par de vraies requêtes HTTP vers une API **json-server** via un service **Axios**.
Gérer les états **isLoading** et **error** dans les vues.

Résultat attendu



Ce qui est fourni

Installation des dépendances

```
npm install axios
npm install -g json-server
```

Fichier **db.json** — à créer à la racine du projet

```
{
  "gadgets": [
    {
      "id": 1,
      "name": "Smartphone XZ",
      "price": 799,
      "image": "/phone.jpg",
      "inStock": true,
      "reviews": []
    },
    {
      "id": 2,
      "name": "Laptop Pro",
      "price": 1299,
      "image": "/laptop.jpg",
      "inStock": false,

```

```
"reviews": []
},
{
  "id": 3,
  "name": "Écouteurs Bluetooth",
  "price": 199,
  "image": "/earbuds.jpg",
  "inStock": true,
  "reviews": []
}
]
```

Lancer json-server

```
json-server --watch db.json --port 3000
```

💡 Lancer json-server dans un terminal séparé pendant que Vite tourne dans un autre.

Structure du projet

```
/vue-gadget-shop
├── src/
│   ├── App.vue
│   ├── router/
│   │   └── index.js
│   ├── services/
│   │   └── gadgetService.js
│   ├── views/
│   │   ├── HomeView.vue
│   │   └── GadgetDetailView.vue
│   ├── components/
│   │   ├── GadgetDisplay.vue
│   │   ├── CartDisplay.vue
│   │   └── ReviewForm.vue
├── public/
│   ├── phone.jpg
│   ├── laptop.jpg
│   └── earbuds.jpg
└── db.json
```

CSS — à ajouter dans `HomeView.vue` et `GadgetDetailView.vue`

```
.loading {
  text-align: center;
  padding: 3rem;
  color: #999;
```

```
font-size: 1.1rem;
}

.error-banner {
  background: #fee2e2;
  color: #c62828;
  border-radius: 10px;
  padding: 1rem 1.5rem;
  margin-bottom: 1.5rem;
  font-size: 0.95rem;
}
```

Contraintes

Dans `src/services/gadgetService.js`

- ☒ Importer `axios`
- ☒ Définir `const API_URL = 'http://localhost:3000'`
- ☒ Exporter un objet `gadgetService` avec les méthodes suivantes :
 - `getAll()` — GET `/gadgets`
 - `getById(id)` — GET `/gadgets/:id`
 - `addReview(gadgetId, review)` — PUT `/gadgets/:id` avec le gadget mis à jour

Dans `HomeView.vue`

- ☒ Supprimer le tableau `gadgets` en dur
- ☒ Ajouter `isLoading` et `error` avec `ref()`
- ☒ Appeler `gadgetService.getAll()` dans `onMounted()`
- ☒ Afficher un indicateur de chargement avec `v-if="isLoading"`
- ☒ Afficher un message d'erreur avec `v-if="error"`
- ☒ Gérer les erreurs avec `try / catch / finally`
- ☒ La logique du panier reste inchangée

Dans `GadgetDetailView.vue`

- ☒ Supprimer le tableau `gadgets` en dur
- ☒ Appeler `gadgetService.getById(id)` dans `onMounted()`
- ☒ Gérer `isLoading` et `error`
- ☒ Modifier `addReview()` pour appeler `gadgetService.addReview()`

`App.vue`, `router/index.js`, `CartDisplay.vue`, `GadgetDisplay.vue`, `ReviewForm.vue`

- ☒ **Aucune modification** — ces fichiers restent identiques au step 11

Règles

- ☒ Tous les appels HTTP passent par `gadgetService.js` — jamais directement dans les vues
- ☒ Utiliser `onMounted()` pour déclencher les appels API

-  Toujours gérer les erreurs avec `try / catch / finally`
-  `isLoading` doit passer à `false` dans le bloc `finally`
-  Pas d'appel `axios` directement dans les vues
-  Pas d'utilisation d'IA pour générer le code

 **Vérification** : Ouvrez l'onglet Réseau (Network) dans les DevTools du navigateur et vérifiez que les requêtes GET s'effectuent bien vers `http://localhost:3000/gadgets`. Ajoutez un avis et vérifiez dans `db.json` que les données sont bien persistées.